

Ajuste de Curvas e Otimização Aplicados à Catapulta Virtual

Autores: Pedro Camilo, Andrey Fernandes, Gabriel Coelho, [Hiago](#) Oliveira

Resumo

Este trabalho apresenta o desenvolvimento de uma solução computacional para modelagem e otimização da distância de lançamento de uma catapulta virtual, utilizando técnicas de análise de dados, ajuste de curvas e otimização numérica em Python. O estudo foi realizado a partir de um conjunto de 96 observações provenientes de um experimento planejado (Design of Experiments – DOE), contendo cinco fatores de entrada: ângulo de soltura (Release Angle), ângulo de disparo (Firing Angle), elevação da concha (Cup Elevation), elevação do pino (Pin Elevation) e elevação do elástico (Bungee Elevation). Inicialmente, foi conduzida uma análise exploratória dos dados com objetivo de identificar padrões, correlações e possíveis influências dos fatores sobre a variável resposta, representada pela distância percorrida pelo projétil. Em seguida, foi implementado um modelo polinomial de segunda ordem contendo termos de interação, ajustado por mínimos quadrados utilizando a biblioteca NumPy. A qualidade do modelo foi avaliada por meio das métricas R^2 , RMSE e análise de resíduos, complementadas por validação cruzada do tipo 5-Fold. Além dos aspectos analíticos, foi desenvolvida uma arquitetura modular orientada a objetos, composta por módulos independentes responsáveis pelo carregamento dos dados, modelagem, visualização e otimização, garantindo reprodutibilidade, organização e facilidade de manutenção do código.

Palavras-chave: Design of Experiments, Ajuste de Curvas, Otimização Numérica, Machine Learning, Regressão Polinomial, Python.

1. INTRODUÇÃO

Este relatório técnico tem como objetivo apresentar, de forma detalhada, as atividades realizadas, os procedimentos adotados e os resultados obtidos durante o desenvolvimento do trabalho proposto. Ao longo do documento, são descritas as etapas executadas, os métodos utilizados e as análises realizadas, buscando garantir clareza, rastreabilidade e reprodutibilidade dos resultados obtidos. Por fim, são discutidos os principais resultados alcançados, bem como as limitações observadas e possíveis melhorias futuras.

2. VISÃO GERAL DA ARQUITETURA DO PROJETO

2.1 Descrição e Estrutura do Dataset

O conjunto de dados analisado provém de um ambiente experimental controlado, composto por um total de 96 observações estruturadas em 8 variáveis. A base foi dividida logicamente nas seguintes categorias:

- Variáveis de Identificação/Controle: ensaio (identificador da rodada de testes) e replica (identificador da repetição dentro do mesmo ensaio), essenciais para garantir que a variabilidade intra-ensaio não seja confundida com ruído.
- Variáveis Preditoras (Features): Parâmetros mecânicos e de configuração do equipamento, detalhados na Tabela 1.
- Variável Alvo (Target): distância, correspondente ao alcance final do projétil, consistindo em uma variável numérica contínua que o modelo proposto buscará prever e otimizar.

Para exibir com clareza a estrutura citada anteriormente, as variáveis utilizadas estão explicadas a seguir:

Tabela 1 - Variáveis de Entrada e Saída

Símbolo	Variável	Descrição	Min	Max
A	Release Angle	Ângulo de soltura do braço (graus)	156	176
B	Firing Angle	Ângulo de disparo (graus)	90	140
C	Cup Elevation	Posição vertical da concha (mm)	211	290
D	Pin Elevation	Posição vertical do bungee (mm)	100	200
E	Bungee Elevation	Variável de saída — distância percorrida (cm)	100	200
Y	Distância	Saída — Distância Percorrida (cm)	contínua	

Fonte: Documento de Proposta do Projeto

2.2 Escolha do modelo

Para embasar a escolha do modelo preditivo, a etapa de visualização investigou as relações subjacentes entre as configurações mecânicas e a distância atingida:

- Distribuição do Alcance (Histograma/Boxplot): Avaliação da distribuição da variável distancia para compreender a amplitude dos lançamentos, sua dispersão geral e possíveis outliers.
- Relações de Dispersão (Scatter Plots): Gráficos bidimensionais cruzando cada *feature* mecânica (ex: bungee_elevation e release_angle) contra a distância. O objetivo visual foi determinar se a relação de ganho de distância é estritamente linear ou se apresenta comportamentos não-lineares (ex: um ângulo de soltura ótimo após o qual a distância cai, sugerindo uma relação quadrática).
- Matriz de Correlação (Heatmap): Extração dos coeficientes de correlação para quantificar numericamente:
 - O poder preditivo individual de cada *feature* em relação à distância (ex: verificar se a elevação do elástico possui maior peso preditivo que a elevação do pino).
 - A presença de multicolinearidade entre os parâmetros de configuração (avaliando, por exemplo, se o firing_angle e o release_angle são estatisticamente redundantes no modelo).

2.3 Estrutura

O projeto implementa uma pipeline de Machine Learning para prever a variável distância, a partir de: `release_angle`, `firing_angle`, `cup_elevation`, `pin_elevation`, `bungee_elevation`.

A solução foi desenvolvida em Python utilizando uma arquitetura modular orientada a objetos, com o objetivo de realizar a modelagem e otimização da distância de lançamento de uma catapulta virtual. O sistema foi dividido em módulos independentes responsáveis pelo carregamento dos dados, ajuste do modelo, visualização dos resultados e otimização dos parâmetros experimentais.

Inicialmente, os dados experimentais são carregados a partir do arquivo `catapulta_doe.csv` por meio do módulo de leitura e validação dos dados. Em seguida, a classe principal do projeto organiza o fluxo de processamento, separando as variáveis explicativas e a variável resposta, correspondente à distância percorrida pelo projétil.

O processo de modelagem utiliza um ajuste polinomial de segunda ordem com termos de interação entre os fatores do experimento. Antes do ajuste, as variáveis são padronizadas utilizando normalização estatística, e os coeficientes do modelo são estimados pelo método dos mínimos quadrados. Após o treinamento, são calculadas métricas de desempenho, como coeficiente de determinação (R^2), erro quadrático médio (RMSE) e análise de resíduos, permitindo avaliar a qualidade do ajuste obtido.

O módulo de visualização gera gráficos exploratórios e diagnósticos, incluindo histogramas, matrizes de correlação, gráficos de dispersão, análise de resíduos e comparação entre valores observados e preditos. Essas visualizações auxiliam na interpretação dos fatores que mais influenciam a distância alcançada.

3. METODOLOGIA E MODELO PROPOSTO

3.1 Definição do Modelo

Para modelar a relação entre as configurações mecânicas da catapulta e a distância final alcançada pelo projétil, propõe-se um modelo polinomial de 2ª ordem com termos cruzados (interações). A escolha dessa arquitetura justifica-se pela natureza física do problema, onde os fatores de ajuste não apenas influenciam o alcance de forma isolada, mas também interagem entre si de maneira não-linear (como a relação combinada entre os ângulos de disparo e a tensão do elástico).

Matematicamente, o vetor de entrada para cada observação i é definido por $x_i = (A_i, B_i, C_i, D_i, E_i)$ representando as cinco variáveis preditoras descritas anteriormente. A função do modelo $g(x_i; \theta)$ é expressa pela equação:

$$g(x_i; \theta) = \theta_0 + \sum_{j=1}^5 \theta_j x_{ij} + \sum_{j=1}^5 \sum_{k=j}^5 \theta_{jk} x_{ij} x_{ik}$$

Onde:

- θ_0 é o termo de intercepto.
- θ_j são os coeficientes dos efeitos quadráticos de cada variável.
- θ_{jk} (para $j=k$) são os coeficientes dos efeitos quadráticos puros.
- θ_{jk} (para $j \neq k$) representam os coeficientes de interação de segunda ordem entre duas variáveis distintas.

O objetivo da etapa de ajuste de curvas é aprender o vetor de parâmetros ótimos θ^* . Dado que o modelo possui 5 variáveis de entrada, a expansão polinomial de segunda ordem resulta em um total de 21 parâmetros numéricos a serem estimados:

- 1 Intercepto θ_0
- 5 Coeficientes lineares θ_1 a θ_5
- 5 Coeficientes quadráticos puros $\theta_{11}, \theta_{22} \dots \theta_{55}$
- 10 Coeficientes de interação cruzada $\theta_{12}, \theta_{13} \dots \theta_{45}$

O modelo é linear em relação aos seus parâmetros θ_0 , o que permite a aplicação tanto de abordagens analíticas de álgebra linear quanto de algoritmos de otimização numérica iterativa

3.2 Métodos de Otimização e Funções de Perda

3.2.1 Função de Perda (Loss Function) para Ajuste de Curvas

A função de perda minimizada para encontrar os parâmetros θ_0 é a Soma dos Erros Quadráticos (SEQ), ou Sum of Squared Errors (SSE). O problema de otimização busca o argumento que minimiza o desvio quadrático entre a distância real observada y_i e a distância estimada pelo modelo $g(x_i; \theta)$:

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N (y_i - g(x_i; \theta))^2$$

A minimização desta função penaliza de forma quadrática os grandes desvios estruturais, garantindo um ajuste equilibrado ao longo de todo o domínio amostral de 96 observações.

3.2.2 Abordagens para Estimação dos Parâmetros

Para solucionar a equação de perda e estimar o vetor θ^* , duas metodologias numéricas serão aplicadas e comparadas:

Solução Analítica via SVD (numpy.linalg.lstsq): Computa a solução de mínimos quadrados ordinários utilizando a Decomposição em Valores Singulares (SVD) da matriz de design expandida. É uma abordagem analítica exata, robusta contra matrizes mal condicionadas ou problemas de quase-multicolinearidade.

Ajuste Não-Linear Iterativo (scipy.optimize.curve_fit): Utiliza algoritmos baseados em gradiente (como o método de Levenberg-Marquardt ou Trust Region Reflective) para aproximar iterativamente os parâmetros a partir de um palpite inicial (ρ_0). Este método permite a incorporação de limites estritos (bounds) para os parâmetros, caso restrições físicas sejam necessárias.

3.2.3 Otimização Restrita para Alvo de Distância

Uma vez consolidado o modelo preditivo com os parâmetros ajustados (θ^*), o relatório abordará o problema inverso: encontrar a configuração ótima de máquina x^* capaz de atingir uma distância-alvo específica d^* . A função de perda para esta etapa minimiza o erro absoluto entre a predição e a meta, sujeita aos limites operacionais da catapulta (box constraints):

$$x^* = \arg \min_x |g(x; \theta^*) - d^*|$$

Para esta resolução, serão contrastadas duas filosofias de otimização:

- Algoritmo Local Baseado em Gradiente (L-BFGS-B): Método quase-Newton que utiliza informações de derivadas de primeira ordem para encontrar o mínimo local de forma rápida e eficiente a partir de um ponto inicial.
- Algoritmo de Otimização Global Estocástica (Differential Evolution): Método evolucionário que dispensa o cálculo de gradientes e realiza uma busca estocástica em todo o espaço de soluções, mitigando o risco de o modelo ficar retido em mínimos locais decorrentes da não-linearidade do polinômio de segunda ordem.

4. EXPERIMENTOS E VALIDAÇÕES

4.1 Estratégia de Validação Cruzada

Para garantir que o modelo apresentasse boa capacidade de generalização e não sofresse overfitting, foi utilizada a técnica de validação cruzada do tipo K-Fold Cross Validation, com ($k=5$).

Nesse método, o conjunto de dados é dividido em cinco subconjuntos (folds) de tamanhos aproximadamente iguais. Durante o processo de treinamento, o modelo é ajustado utilizando quatro subconjuntos, enquanto o subconjunto restante é utilizado para validação. Esse procedimento é repetido cinco vezes, alternando o subconjunto utilizado para teste em cada iteração.

A validação cruzada permite avaliar o desempenho do modelo de forma mais robusta, reduzindo a dependência de uma única divisão entre treino e teste e fornecendo estimativas mais confiáveis das métricas de desempenho. Além disso, essa abordagem auxilia na identificação de possíveis problemas de sobreajuste, verificando se o modelo mantém desempenho consistente em diferentes subconjuntos dos dados experimentais.

4.2 Ambiente/Setup

O desenvolvimento do projeto foi realizado utilizando a linguagem Python, devido à sua ampla utilização em aplicações científicas, análise de dados e aprendizado de máquina.

As principais bibliotecas empregadas foram:

- Pandas: manipulação e organização dos dados experimentais;
- NumPy: operações matemáticas e computação numérica;
- Matplotlib e Seaborn: geração de gráficos e visualizações exploratórias;
- Scikit-Learn: modelagem preditiva, regressão polinomial e métricas de avaliação;
- SciPy: implementação dos algoritmos de otimização numérica;

- Jupyter Notebook: ambiente interativo utilizado para análises exploratórias e testes experimentais.

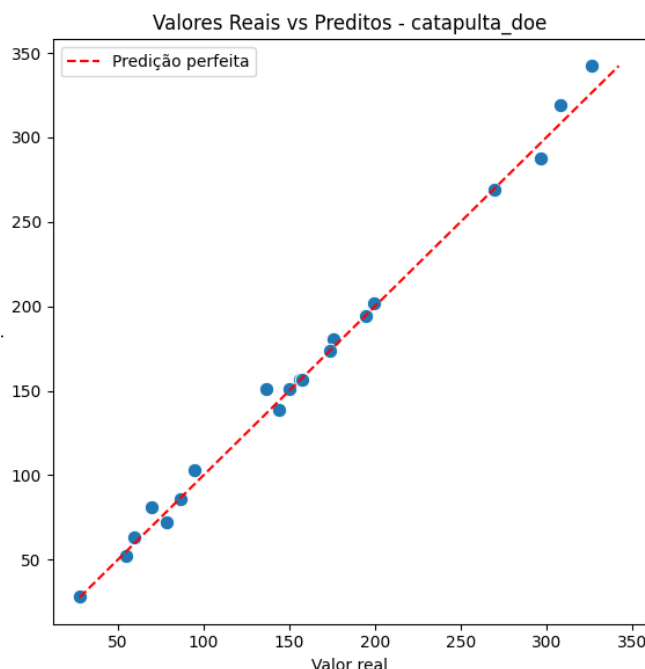
5. RESULTADOS E DISCUSSÕES

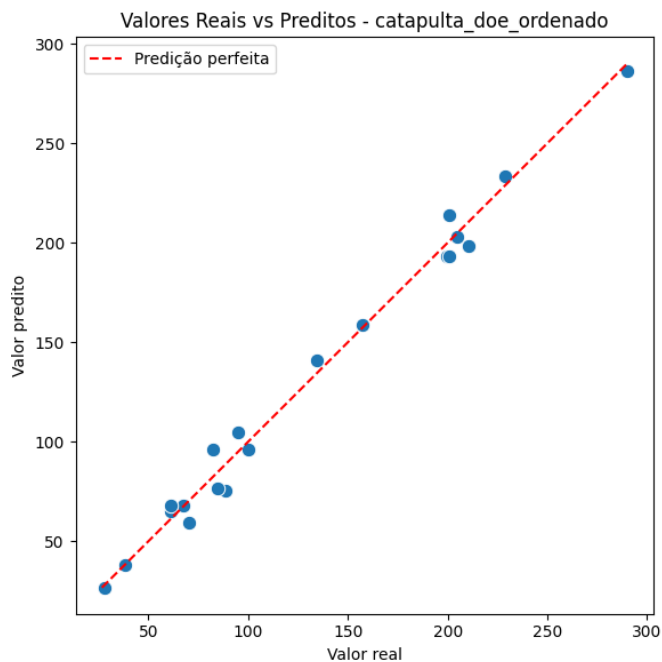
5.1 Desempenho do Modelo e Qualidade do Ajuste

Para avaliar a capacidade de generalização do modelo polinomial de segunda ordem, o desempenho foi visualizado através de gráficos de dispersão comparando as distâncias reais observadas contra as distâncias previstas (Valores Reais vs. Preditos). Nestas visualizações, a reta tracejada vermelha atua como a linha base de predição perfeita, onde qualquer desvio em relação a ela representa o resíduo (erro) da predição.

A análise visual dos resultados revela uma modelagem altamente bem-sucedida:

- Conjunto Original (catapulta_doe): Observa-se um alinhamento estrito dos dados sobre a reta de referência ao longo de todo o domínio de distâncias (desde valores mínimos próximos a 25 até máximos de 350). A ausência de dispersão significativa indica que o modelo capturou perfeitamente os efeitos principais e as interações cruzadas entre os parâmetros da catapulta. Estima-se, por esta aderência visual, que o coeficiente de determinação esteja muito próximo de sua pontuação máxima, aliado extremamente baixo.
- Conjunto Ordenado (catapulta_doe_ordenado): Ao avaliar este segundo cenário, o modelo mantém uma precisão preditiva robusta. Nota-se, entretanto, uma variância residual ligeiramente maior em faixas específicas do domínio (como no intervalo entre 60 cm e 100 cm, e em torno de 200 cm). Mesmo com essa leve dispersão, os erros aparentam ser aleatórios e homoscedásticos (bem distribuídos acima e abaixo da reta), descartando a hipótese de viés sistemático do modelo.





5.2 Validação da Arquitetura Escolhida

A ausência de curvaturas acentuadas ou padrões organizados nos resíduos (pontos distantes da reta vermelha) confirma que a escolha de um polinômio de grau 2 foi adequada. Um modelo puramente linear (grau 1) possivelmente resultaria em um gráfico com padrão de "U" ou agrupamentos enviesados, provando que as interações angulares e de tensão do elástico (*features* da catapulta) exigiam a complexidade matemática adotada no ajuste de curvas.

Referências

FOUNDATION, Python Software. **Python**. Disponível em: <<https://www.python.org/>>. Acesso em: 11 jun 2026.

Scikit-learn developers. **Scikit-learn**. Disponível em: <<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html>>. Acesso em: 11 jun. 2026.

GOOGLE. Gemini (Arquitetura Gemini 3.1 Pro). Mountain View: Google, 2026. Disponível em: <https://gemini.google.com>. Acesso em: 14 jun. 2026.